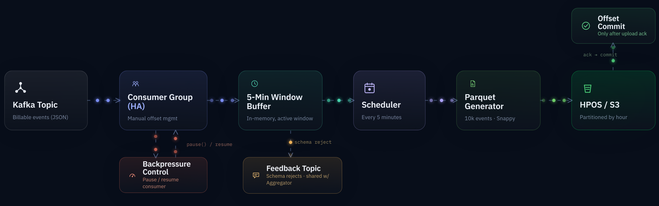
EVENT PROCESSOR · LOW-LEVEL DESIGN

ZERO-LOSS

Ingestion & parquet pipeline

High-throughput service: consumes Kafka events, buffers in 5-minute windows, generates parquet files, persists to HPOS/S3, and guarantees zero event loss.



LOW-LEVEL DESIGN

Design detail

ARCHITECTURE OVERVIEW

Event Processor is a **single service**. It continuously consumes events from Kafka, stores them in time-window based buffers, generates parquet files, uploads parquet files to HPOS, and commits Kafka offsets only after successful persistence. **Event Processor performs lightweight schema validation only** — business and billing rules are intentionally deferred to the **Aggregation Layer**. Schema-invalid events are routed to a feedback topic and are not written to parquet.

Consumer Design

- **Consumer group** provides high availability — multiple instances run under the same group.
- Kafka partitions are distributed automatically across instances.
- **Manual offset management** — offsets are never committed at message-receipt time.
- Offsets are committed only after parquet files upload successfully to HPOS.

Buffering Strategy

- Events buffered in memory using **5-minute ingestion-time windows**.
- Consumer writes only to the **active window**; scheduler processes only closed windows.
- Avoids synchronization conflicts between consumer and scheduler threads.

`10:00–10:04:59` `10:05–10:09:59` `10:10–10:14:59`

Consumer Group Configuration

```
group.id = event-processor
enable.auto.commit = false
auto.offset.reset = earliest
isolation.level = read_committed
max.poll.records = 1000
max.poll.interval.ms = 600000
partition.assignment.strategy = cooperative-sticky
```

Offsets are committed manually via `commitSync()` only after the HPOS upload ack — never at message receipt.

Capacity & Sizing

Proposed load · 2 billion events / month

772 /sec Average throughput	**~3,000** /sec Planned peak (~4×)
12 Kafka partitions	**3–4** last Scale to 12 on backlog

≈ 231k events per 5-min window ≈ 23 parquet files — consistent with the 250k → 25 example. Each instance owns 3–4 partitions (N+1 HA).

Scheduler Design `executes every 5 minutes`

Identifies closed windows and initiates parquet generation. Maintains a **processing checkpoint table** tracking each window through its lifecycle.

STATE TRANSITIONS

OPEN → CLOSED → PROCESSING → STORED → COMPLETED

CHECKPOINT TABLE

WindowId	Status	StartTime	EndTime	ParquetCount	UploadStatus
w_1015	COMPLETED	10:15:00	10:19:59	7	OK
w_1020	PROCESSING	10:20:00	10:24:59	3	…

Parquet Generation

- Target size **10,000 events** per parquet file.
- File names include **date/hour, partition and offset range** — to support replay and idempotency.
- Events are grouped by **event-time hour** — late events land as a separate file in their own `hour=` partition, so one window may write to more than one hour folder.
- Compression uses **Snappy** for efficient storage and downstream processing.

```
HPOS  s3://bucket/year=2026/month=07/day=09/hour=18/
file  events_2026-07-09_h18_p1_off_120000-129999.parquet
```

WORKED EXAMPLE

250,000
events in one window

↓

25 files
at 10,000 events each

Offset Commit Strategy

- After parquet creation, HPOS upload is initiated.
- Offsets are committed **only when upload acknowledgement is received for all files** in the window.
- If upload fails, offsets remain uncommitted and Kafka replay guarantees recovery — **zero data loss**.

HPOS Failure Handling

No separate retry queue — Kafka is the durable buffer. Uploads are retried in-process with backoff; if failures persist, the consumer is paused.

BACKOFF SCHEDULE

`1 min` `5 min` `15 min` `30 min`

Escalation: backoff exhausted → `consumer.pause()` → Kafka replay on recovery; offsets stay uncommitted (**zero loss**).

Retry metadata: attempt count · first failure timestamp · last retry timestamp.

Backpressure Handling

- If the scheduler falls behind, memory consumption may grow.
- Define thresholds for **max events buffered** and **memory utilization**.
- On breach, consumers are paused via `consumer.pause()`; consumption resumes automatically after the backlog decreases.

Late Arriving Events

- Consumer writes only to the current **active (ingestion-time) window** — buffering stays simple.
- At generation the scheduler **groups events by event-time hour**. A late event (e.g. `11:xx` arriving at `14:xx`) is written as a separate file into `hour=11`; the rest into `hour=14`.
- Existing files are never mutated — the late file is an **append**, named by offset range so replay stays idempotent. Offsets commit only after **all** files in the window upload.
- Aggregation Layer re-reads recent partitions within its **grace window** and owns all late-arrival business logic.

Feedback Topic & Schema Validation `shared with Aggregation Layer`

- Event Processor performs **schema validation only** (structural / deserialization). Business & mandatory-parameter rules stay in the Aggregation Layer.
- Schema-invalid events (bad requests, malformed payloads) are published to the **feedback topic** and are **not** written to parquet.
- The same topic is reused by the **Aggregation Layer** to publish per-event success / failure status — one shared reconciliation stream.
- Published with an **idempotent producer** (acks=all); the source offset commits only after the feedback publish succeeds — preserving zero loss.

FEEDBACK MESSAGE

```
{
"eventId": "evt-8f2a…",
"stage": "EVENT_PROCESSOR",
"status": "REJECTED",
"reasonCode": "SCHEMA_INVALID",
"message": "missing field: amount",
"ts": "2026-07-09T18:04:12Z"
}
```

Optional — DLQ not required for v1. *If malformed payloads must be reprocessed later, write raw rejected bytes to an HPOS `rejected/` prefix rather than a dedicated topic.*

Failure Scenarios

1 Consumer crash before upload
Kafka replay recovers events — offsets were never committed.

2 Crash after upload, before commit
Idempotent parquet naming prevents duplicates on replay.

3 HPOS unavailable
Retries with backoff; if it persists the consumer is paused. Offsets are not committed until success.

4 Kafka rebalance
Partitions are reassigned automatically across the consumer group.

High Availability

- Deploy **multiple Event Processor instances**.
- Health checks, auto-restart policies, and Kafka consumer groups.
- No shared state between instances — except Kafka offsets and HPOS persistence.

Observability

METRICS

`consumer lag` `throughput/sec` `buffer size` `memory usage` `parquet files generated` `upload latency` `retry count` `paused-consumer duration` `upload failures` `scheduler execution duration`

ALERTS

`consumer lag` `memory thresholds` `repeated upload failures`